Scripts en Linux

1 STRUCTURE DE BASE D'UN SCRIPT

```bash
#!/bin/bash
# Toujours commencer par cette ligne (Shebang)
```

Rendre exécutable :

```bash
chmod +x nom_script.sh
./nom_script.sh
```

2 LES VARIABLES

Déclaration et utilisation

```bash
NOM="valeur"          # Déclaration (pas d'espace autour de =)
echo $NOM             # Affichage
echo "${NOM}"         # Affichage sécurisé
```

Variables spéciales

Variable	Signification
`$0`	Nom du script
`$1`, `$2`…	Arguments du script
`$#`	Nombre d'arguments
`$?`	Code de retour de la dernière commande (0 = succès)
`$$`	PID du processus actuel

3 LES TESTS CONDITIONNELS

Syntaxe de base

```bash
if [ condition ]; then
    commandes
elif [ autre_condition ]; then
    commandes
else
    commandes
fi
```

Tests sur les FICHIERS

Test	Signification
`[ -f fichier ]`	Fichier existe (fichier régulier)
`[ -d repertoire ]`	Répertoire existe
`[ -e fichier ]`	Fichier ou répertoire existe
`[ -r fichier ]`	Fichier lisible
`[ -w fichier ]`	Fichier inscriptible
`[ -x fichier ]`	Fichier exécutable
`[ -s fichier ]`	Fichier existe et n'est pas vide

Tests sur les CHAÎNES

Test	Signification
`[ -z "$VAR" ]`	Variable vide
`[ -n "$VAR" ]`	Variable non vide
`[ "$A" = "$B" ]`	Égalité
`[ "$A" != "$B" ]`	Différence

Tests NUMÉRIQUES

Test	Signification
`[ $A -eq $B ]`	Égal (equal)
`[ $A -ne $B ]`	Différent (not equal)
`[ $A -lt $B ]`	Inférieur (less than)
`[ $A -le $B ]`	Inférieur ou égal
`[ $A -gt $B ]`	Supérieur (greater than)
`[ $A -ge $B ]`	Supérieur ou égal

Opérateurs logiques

bash

```bash
[ cond1 ] && [ cond2 ]    # ET logique
[ cond1 ] || [ cond2 ]    # OU logique
! [ condition ]           # NON logique
```

4 LES BOUCLES

Boucle WHILE (la plus utilisée)

```bash
while [ condition ]; do
    commandes
done
```

Lecture d'un fichier ligne par ligne :

```bash
while IFS= read -r ligne; do
    echo "Ligne : $ligne"
done < /chemin/fichier.txt
```

Boucle FOR

```bash
# Sur une liste
for i in 1 2 3 4 5; do
    echo $i
done

# Sur les fichiers d'un répertoire
for fichier in *.txt; do
    echo "Fichier : $fichier"
done

# Boucle C-style
for ((i=0; i<10; i++)); do
    echo $i
done
```

Boucle UNTIL

```bash
until [ condition ]; do
    commandes
done
```

5 COMMANDES ESSENTIELLES

Gestion des dates

```bash
date                    # Date complète
date +%d-%m-%Y          # Format personnalisé
# %d = jour, %m = mois, %Y = année (4 chiffres), %y = année (2 chiffres)
# %H = heure, %M = minutes, %S = secondes
```

Manipulation de chemins

```bash
basename /var/www       # Affiche : www
dirname /var/www        # Affiche : /var
```

Archivage avec TAR

```bash
# Créer une archive compressée
tar -czf archive.tar.gz /chemin/dossier

# Options importantes :
# -c : créer
# -x : extraire
# -z : compresser avec gzip
# -f : spécifier le fichier
# -v : verbeux (affiche les fichiers)
# -C : changer de répertoire avant opération

# Exemple avec -C
tar -czf backup.tar.gz -C /var www
```

Transfert avec RSYNC

bash

```bash
rsync -avz source/ user@IP:destination/

# Options :
# -a : mode archive (préserve permissions, dates...)
# -v : verbeux
# -z : compression pendant transfert
# --delete : supprime fichiers absents à la source
```

Autres commandes utiles

bash

```bash
mkdir -p /chemin        # Crée répertoire + parents
cp -r source dest       # Copie récursive
rm -rf dossier          # Suppression forcée et récursive
chmod 755 fichier       # Change permissions
chown user:group fichier # Change propriétaire
```

6 Lecture/Écriture de FICHIERS

Lecture

bash

```bash
# Méthode 1 : while read
while read ligne; do
    echo $ligne
done < fichier.txt

# Méthode 2 : cat dans une boucle
cat fichier.txt | while read ligne; do
    echo $ligne
done
```

Écriture

bash

```bash
echo "texte" > fichier.txt      # Écrase le fichier
echo "texte" >> fichier.txt     # Ajoute à la fin
commande > fichier.txt          # Redirection stdout
commande 2> erreur.txt          # Redirection stderr
commande &> fichier.txt         # Redirection stdout + stderr
```

7 GESTION DES ERREURS

Tester le code de retour

```bash
commande
if [ $? -eq 0 ]; then
    echo "Succès"
else
    echo "Échec"
fi
```

Ou plus court

```bash
commande || echo "Erreur lors de la commande"
commande && echo "Commande réussie"
```

Sortie du script

```bash
exit 0      # Succès
exit 1      # Erreur générale
exit 2      # Mauvaise utilisation
```

8 FONCTIONS

bash

```bash
ma_fonction() {
    echo "Paramètre 1 : $1"
    echo "Paramètre 2 : $2"
    return 0    # Code de retour
}

# Appel
ma_fonction "arg1" "arg2"
```

9 CASE (Menu choix multiple)

bash

```bash
case $variable in
    valeur1)
        commandes
        ;;
    valeur2)
        commandes
        ;;
    *)
        commandes par défaut
        ;;
esac
```

🔟 PIÈGES À ÉVITER (IMPORTANT !)

✅ Toujours mettre des guillemets

```bash
# ❌ MAUVAIS
if [ -f $fichier ]; then

# ✅ BON
if [ -f "$fichier" ]; then
```

✅ Espaces autour des crochets

```bash
# ❌ MAUVAIS
if [$a -eq $b]; then

# ✅ BON
if [ $a -eq $b ]; then
```

✅ Pas d'espace autour de =

```bash
# ❌ MAUVAIS
VAR = "valeur"

# ✅ BON
VAR="valeur"
```

✅ Vérifier les arguments

```bash
if [ $# -eq 0 ]; then
    echo "Usage : $0 <argument>"
    exit 1
fi
```